



ANTEPROYECTO DEL TRABAJO DE FIN DE GRADO

Alumno/a	David Muñoz del Valle				
Titulación:	Ingeniería del Software				
Tutor/es:	Gabriel Luque Polo				
Título	Librería extensible de optimización metaheurística en Rust				
Subtítulo <i>(solo si en grupo)</i>					
Título en inglés	Extensible metaheuristic optimization library in Rust				
Subtítulo en inglés <i>(solo si en grupo)</i>					
Trabajo en grupo:	Sí	☐	No	☒	
Otros integrantes del grupo:					

Contextualización del problema a resolver. Describir claramente de dónde surge la necesidad de este TFG y el dominio de aplicación. En caso de que el TFG se base en trabajos previos, debe aclararse cuáles son las aportaciones del TFG.

El desarrollo de este Trabajo de Fin de Grado surge ante la escasez de librerías de optimización metaheurística en el ecosistema Rust [3]. Actualmente existen pocas librerías de este tipo y, en general, presentan limitaciones en cuanto a madurez, extensibilidad y variedad de algoritmos disponibles.

Esta situación dificulta el uso de Rust en proyectos que requieren técnicas de optimización metaheurística, a pesar de las ventajas que ofrece el lenguaje en términos de rendimiento y seguridad. Como respuesta a esta necesidad, este TFG propone el diseño y desarrollo de una librería de optimización metaheurística extensible en Rust, que permita ampliar fácilmente el conjunto de algoritmos y adaptarse a distintos tipos de problemas.

El trabajo se apoya en soluciones previas existentes, tanto en Rust como en otros lenguajes, pero aporta como contribución principal una arquitectura modular y orientada a la extensibilidad, orientada a facilitar el desarrollo y la experimentación con metaheurísticas dentro del ecosistema Rust.

Descripción detallada de en qué consistirá el TFG. En caso de que el objeto principal del TFG sea el desarrollo de software, además de los objetivos generales deben describirse sus funcionalidades a alto nivel.

El proyecto pretende ser una forma de ayudar a los desarrolladores a implementar algoritmos de optimización metaheurística en un lenguaje moderno como es Rust.

Ya existen librerías de este estilo, pero no hay muchas y están limitadas, la idea es que esta pueda servir a un público mayor, debido a que pretende ser extensible.

El presente Trabajo de Fin de Grado consiste en el diseño y desarrollo de una librería de optimización metaheurística en Rust, cuyo objetivo principal es facilitar a los desarrolladores la implementación y el uso de este tipo de algoritmos en un lenguaje moderno, seguro y eficiente.

Aunque existen algunas librerías de optimización metaheurística en Rust, su número es reducido y presentan limitaciones en cuanto a funcionalidad y capacidad de ampliación. En este contexto, el proyecto propone el desarrollo de una librería orientada a un público amplio, con un diseño modular que permita su extensión mediante la incorporación de nuevos algoritmos, operadores y tipos de problemas, sin necesidad de modificar el núcleo del sistema.

A alto nivel, la librería ofrecerá una arquitectura común para la definición de problemas de optimización y la implementación de distintas metaheurísticas. Asimismo, se proporcionarán mecanismos básicos para la configuración de los algoritmos, la ejecución de experimentos y la obtención de resultados, junto con documentación y ejemplos de uso que faciliten su adopción por parte de otros desarrolladores.

Listado de resultados que generará el TFG (aplicaciones, estudios, manuales, etc.)

Software, Librería Rust

Documentación

Pruebas

METODOLOGÍA:

Descripción de la metodología empleada en el desarrollo del TFG. Especificar cómo se va a desarrollar. Concretar si se trata de alguna metodología existente y, en caso contrario, describir y justificar adecuadamente los métodos que se aplicarán.

Para el desarrollo de la librería se empleará una metodología iterativa e incremental, inspirada en metodologías ágiles y, en particular, en un enfoque similar a SCRUM [6], adaptado al desarrollo individual de una única persona.

El proceso comenzará con una fase inicial de análisis de requisitos y planificación temporal, en la que se definirán los objetivos del trabajo y se dividirá el desarrollo en un conjunto de iteraciones. Cada iteración tendrá una fecha estimada de inicio y finalización, así como un conjunto de funcionalidades a implementar.

Debido a la naturaleza y a las características propias del desarrollo de software, esta metodología se plantea como flexible y adaptable al cambio. Por ello, las iteraciones estarán abiertas a modificaciones en cuanto a duración, alcance o prioridades, siendo completamente normal la aparición de retrasos, adelantos o ajustes en la planificación inicial.

Asimismo, se contempla la posibilidad de que durante el desarrollo surjan requisitos incompletos, inconsistentes o no previstos inicialmente, los cuales se irán refinando y ajustando de forma progresiva conforme avance el proyecto. Este enfoque permite integrar mejoras continuas, realizar refactorizaciones cuando sea necesario y adaptar la arquitectura de la librería para favorecer su extensibilidad y mantenibilidad.

En cada iteración se abordarán las fases de diseño, implementación, pruebas y documentación, garantizando así una validación continua del software desarrollado y una evolución controlada del proyecto hasta su finalización

FASES DE TRABAJO:

Enumeración y breve descripción de las fases de trabajo en las que consistirá el TFG.

1. Estudio

a. Metaheurística:

Análisis de algoritmos (Genéticos, Recocido Simulado, búsqueda tabú, etc.) para identificar abstracciones comunes.

b. Lenguaje de programación, Rust:

c. Compilador, Cargo

d. Crates

e. Estado del Arte:

Estudio de librerías existentes para identificar oportunidades de mejora; jMetal [1], metaheuristics [2], optim.lib [5].

2. Organización, metodología

a. Análisis de requisitos

b. División del trabajo en iteraciones:

Adopción de una metodología ágil para la división del trabajo en iteraciones (Sprints), permitiendo entregas incrementales de código funcional.

3. Arquitectura

a. Diseño de la arquitectura:

Definición de los componentes principales (Solución, Problema, Algoritmo, Observadores).

b. Decisión sobre los patrones de diseño:

Selección de patrones (ej. Strategy para operadores, Builder para configurar algoritmos) que aprovechen el sistema de Traits de Rust para permitir que el usuario añada sus propias lógicas.

4. Desarrollo

a. Código fuente, pruebas, ejemplos:

Implementación del núcleo de la librería y las metaheurísticas base, junto al desarrollo de tests unitarios y de integración para asegurar la robustez (aprovechando cargo test) y la creación de casos prácticos (ej. Problema del Viajante o Mochila) para demostrar la facilidad de uso y la extensibilidad.

5. Conclusiones

a. Obtención de métricas:

Pruebas de rendimiento (benchmarks) y comparación de resultados con otras librerías o soluciones conocidas.

b. Análisis de los resultados

c. Obtención de las conclusiones

d. Identificación de las limitaciones:

Reflexión sobre los puntos fuertes del lenguaje y las dificultades encontradas.

6. Documentación y memoria

a. Desarrollo de la documentación

b. Desarrollo de la memoria:

Elaboración del documento académico que recoge todo el proceso.

TEMPORIZACIÓN:

La siguiente tabla deberá contener una fila por cada una de las fases enumeradas en la sección anterior. La temporización **no debe incluir las horas dedicadas a la preparación de la presentación y defensa del TFG**.

En caso de tratarse de un **trabajo en grupo**, se añadirá una columna HORAS por cada miembro del equipo. Debe especificarse claramente el número de horas dedicado por cada alumno/a y la suma de horas individual deberá ser también de 296.

FASE	HORAS
	David Muñoz del Valle
Estudio sobre Metaheurística	16
Estudio sobre el lenguaje de programación (RUST)	25
Estudio sobre el compilador, Cargo	4
Estudio sobre los Crates	2
Estado del arte	8



Análisis de los requisitos	7
División del trabajo en iteraciones	3
Diseño de la arquitectura	28
Decisión sobre los patrones de diseño	7
Desarrollo de la librería	50
Implementación paralela	15
Desarrollo de las pruebas	20
Desarrollo de los observables	10
Desarrollo de ejemplos	7
Obtención de métricas	10
Análisis de los resultados	2
Obtención de las conclusiones	2
Identificación de las limitaciones	2
Desarrollo de la documentación	23
Desarrollo de licencia, consideraciones legales y configuración del repositorio	5
Desarrollo de la memoria	50
	296

TECNOLOGÍAS EMPLEADAS:

Enumeración de las tecnologías utilizadas (lenguajes de programación, frameworks, sistemas gestores de bases de datos, etc.) en el desarrollo del TFG.

Lenguaje de programación: Rust

Gestión de paquetes y compilación: Cargo

Control de versiones: Git

Repositorio remoto: GitHub

RECURSOS SOFTWARE Y HARDWARE:

Listado de dispositivos (placas de desarrollo, microcontroladores, procesadores, sensores, robots, etc.) o software (IDE, editores, etc.) empleados en el desarrollo del TFG.

Software:

IDE/Editor de código: Visual Studio Code

IDE/Editor de código: Rust Rover

Extensiones Visual Studio Code: Rust Analyzer

Sistema operativo: Arch Linux (64-bit)

Hardware, ordenador personal:**Procesador (CPU):**

AMD Ryzen 5 5500U, Radeon

Arquitectura: x86_64 (32-bit y 64-bit)

Núcleos: 6

Hilos: 12 (2 hilos por núcleo)

Frecuencia máxima: 4,06 GHz

Caché: L1 192 KiB, L2 3 MiB, L3 8 MiB

Virtualización: AMD-V

Memoria RAM: 7,1 Gb

Tarjeta gráfica: AMD Lucienne, integrada en CPU

Listado de referencias (libros, páginas web, etc.)

[1] JMetal

J.J. Durillo, A.J. Nebro [jMetal: a Java Framework for Multi-Objective Optimization](#). Advances in Engineering Software 42 (2011) 760-771.

[2] metaheuristics

Shah, A. S. (2025). MetaheuRUSTics: A comprehensive collection of metaheuristic optimization algorithms implemented in Rust. <https://github.com/arvashah2k/metaheuRUSTics>

[3] Rust

<https://rust-lang.org/es/>

[4] Cargo

<https://doc.rust-lang.org/cargo/>

[5] optim.lib

<https://optimlib.readthedocs.io/en/latest/>

[6] SCRUM

<https://www.scrum.org/resources/blog/que-es-scrum>

Málaga, _____ de _____ de _____



UNIVERSIDAD
DE MÁLAGA



Firma tutor/tutora:

Firma cotutor/a:

Firma tutor/a coordinador/a: